



Smart Farm IoT Monitoring and Alert System

Project Report

Internet of Things 2025/2026

University of Cagliari

Supervised by: Prof. Michele Nitti

Students:

Ghita El Belghiti

Essil Amaimi

Table of contents

1. Introduction	3
2. Planning Phase	3
2.1 System Request and Motivation	3
2.2 Feasibility Analysis	3
3. Analysis Phase	4
3.1 Reference Scenario	4
3.2 State of the Art Competitor Analysis	4
3.3 Functional Requirements	5
3.4 Non-Functional Requirements	7
3.5 Technology Analysis	7
3.5.1 Hardware Selection	7
3.5.2 Communication Protocol	7
3.5.3 User Interface	8
4. Design Phase	9
4.1 System Architecture	9
4.2 Database Design	9
4.3 Data Exchange Format	9
4.4 Hardware Wiring Design	10
4.5 Software Flowcharts	10
4.5.1 ESP8266 Data Collection and Transmission (FR1, FR2)	10
4.5.2 Python MQTT Subscriber (FR3)	10
4.5.3 Dashboard Alert Algorithm (FR6)	10
4.6 Digital Twin Concept	11
4.7 Dashboard Mock-up	11
5. Implementation Phase	12
5.1 System Construction	12
5.1.1 Hardware Implementation	12
5.1.2 Software Implementation	12
5.2 Testing	12
5.3 Known Limitations and Future Work	13
6. Conclusion	14

1. Introduction

This report presents the design, implementation, and evaluation of a Smart Farm IoT Monitoring System developed as part of the *Internet of Things* course. The project follows the **System Development Life Cycle (SDLC)** methodology, covering the analysis, design, implementation, integration, and testing phases.

The proposed system enables real-time monitoring of environmental conditions affecting plant growth by collecting data from multiple sensors connected to an ESP8266 microcontroller. The measured parameters, including temperature, humidity, soil moisture, and light intensity, are transmitted using the MQTT communication protocol to a local server, where they are stored in a SQLite database. A Flask-based web application provides a real-time dashboard for data visualization, historical analysis, and system monitoring.

To improve usability, the system incorporates automatic alert generation based on predefined environmental thresholds and sends notifications to users through a Telegram bot whenever abnormal conditions are detected. In addition, the system supports the remote activation of an actuator through a relay module, allowing users to remotely trigger an irrigation device or another external load.

The project was developed collaboratively by two students. Both team members contributed to the hardware and software development throughout the project, including the ESP8266 firmware, MQTT communication, database integration, Flask backend, web dashboard, system testing, and feature integration. Development responsibilities evolved during the implementation phase, with both members actively participating in the design, implementation, debugging, and refinement of the final system.

2. Planning Phase

2.1 System Request and Motivation

Agriculture is one of the sectors where IoT can make the biggest difference. Small farms and greenhouses often rely on manual checks to monitor environmental conditions like temperature, humidity, soil moisture, and light. This is not only time-consuming but also unreliable, since conditions can change quickly and go unnoticed.

The idea behind this project is to build a low-cost smart farming system that automates environmental monitoring while also enabling remote interaction with the monitored environment. The system continuously collects sensor data and presents it through a web dashboard, allowing farmers or researchers to monitor plant conditions from any web browser without being physically present. In addition to monitoring, the system enables users to remotely activate an actuator connected to the ESP8266 through a relay module, making it possible to simulate or control irrigation when environmental conditions require intervention.

2.2 Feasibility Analysis

Technical feasibility: The ESP8266 is a widely available and well-documented microcontroller with built-in WiFi, which makes it suitable for this kind of project. The sensors selected (DHT11, soil moisture sensor, LDR) are inexpensive and compatible with the board. The software stack (Python, Flask, SQLite, MQTT) is open source and runs on any laptop.

Economic feasibility: The hardware cost is very low. The ESP8266 and sensors can be purchased for less than 10 euros in total. The software stack is entirely free and open source.

Organizational feasibility: The system is designed to be simple to deploy. The dashboard is accessible from any browser without installing anything on the client side.

3. Analysis Phase

3.1 Reference Scenario

The scenario we are targeting is a small agricultural environment, such as a greenhouse or an experimental farm plot. The people involved are students or researchers who need to monitor plant growth conditions without always being physically present.

The environment is indoor or semi-outdoor. The sensors need to track four key parameters: temperature, air humidity, soil moisture, and light intensity. These are the main environmental variables that affect plant health and irrigation needs.

The system is expected to be used continuously, logging data every few seconds, and the dashboard should be accessible from any device connected to the local network. Alerts should be generated automatically when values go outside acceptable ranges.

Assumptions made for this project:

- The ESP8266 and the server machine are connected to the same WiFi network.
- The MQTT broker (Mosquitto) runs locally on the server machine.
- No cloud connectivity is required, the system works entirely on a local network.
- A single ESP8266 node is used, monitoring one location.
- Data is stored locally in a SQLite file, no external database server needed.

3.2 State of the Art: Competitor Analysis

Before defining our system, we looked at existing solutions in the smart farm monitoring space to understand what already exists and what we could do differently.

System	Key Features	Advantages	Disadvantages
Blynk IoT Platform	Cloud-based dashboard, mobile app, supports multiple boards	Easy to set up, good UI, mobile access	Requires internet, paid plan for more than 2 devices, data stored on third-party servers
Node-RED + InfluxDB + Grafana	Visual flow editor, time-series database, powerful charts	Very flexible, open source, good for complex setups	Complex to configure, steep learning curve, heavy stack for small projects
ThingSpeak	Cloud IoT platform with MATLAB analytics, free tier available	Simple API, free basic use, built-in chart tools	Limited to 15 second update interval on free plan, data on external server, less customizable

From this analysis, we can identify two main categories of existing systems: **cloud-based platforms** (Blynk, ThingSpeak) which are easy to use but depend on external servers and internet connectivity, and **self-hosted stacks** (Node-RED) which are very powerful but overly complex for a simple monitoring use case.

Our system sits in between: it is fully local like a self-hosted solution, but much simpler to deploy. It does not require internet access, all data stays on the local machine, and the dashboard is custom-built to show exactly what is needed without unnecessary complexity.

3.3 Functional Requirements

The functional requirements describe what the system must be able to do.

FR1 — Sensor Data Acquisition	
Description	The system is able to collect temperature, humidity, soil moisture, and light intensity readings from sensors connected to the ESP8266 microcontroller.
Input	Physical sensor measurements from DHT11, soil moisture sensor, and LDR.
Output	Raw sensor values (float) available for processing and transmission.

FR2 — Data Transmission via MQTT	
Description	The system is able to transmit sensor readings from the ESP8266 to the server using the MQTT protocol over WiFi.
Input	Raw sensor values from FR1.
Output	MQTT messages published to topics: farm/temperature, farm/humidity, farm/moisture, farm/light.

FR3 — Data Storage	
Description	The system is able to store incoming sensor data persistently in a local database so that historical data is available for analysis.
Input	MQTT messages received by the Python subscriber from FR2.
Output	Records inserted into the SQLite database with topic, value, and timestamp.

FR4 — Real-Time Dashboard Visualization	
Description	The user is able to view the current sensor readings and live-updating charts on a web dashboard without refreshing the page manually.
Input	GET request to /api/latest endpoint.
Output	Live KPI cards and charts updated every 5 seconds showing current temperature, humidity, moisture, and light values.

FR5 — Historical Data Visualization	
Description	The user is able to view historical sensor data filtered by date range on a dedicated history page with statistical summaries.
Input	Date range and sensor filter parameters sent to /api/history endpoint.
Output	Historical charts and average/min/max statistics for the selected period.

FR6 — Threshold-Based Alert Generation	
Description	The system is able to generate visual alerts on the dashboard when sensor values exceed predefined safety thresholds.
Input	Current sensor values from FR4 compared against threshold constants defined in the dashboard.
Output	Colored alert banners and Farm Status indicator (Healthy / Monitor Closely / Attention Required).

FR7 — Telegram Notification Service	
Description	The system is able to notify the user through a Telegram bot whenever predefined environmental thresholds are exceeded, allowing remote monitoring without continuously accessing the web dashboard.
Input	Alert conditions generated from the current sensor readings (high temperature, low soil moisture, abnormal humidity, or excessive light intensity).
Output	A Telegram message containing the current sensor values and a description of the detected condition is delivered to the user's mobile device.

FR8 — Remote Actuator Control	
Description	The system is able to remotely activate or deactivate an actuator connected to the ESP8266 through a relay module, allowing the user to perform corrective actions such as starting or stopping an irrigation system
Input	A remote control command issued by the user (through the web interface).
Output	The relay changes its state (ON/OFF), activating or deactivating the connected actuator, while the updated status is reflected in the system.

3.4 Non-Functional Requirements

ID	Category	Description
NFR1	Update Latency	The dashboard must refresh sensor data within 5 seconds of a new reading being published by the ESP8266.
NFR2	Reliability	The system must continue to function and display data even if the ESP8266 is temporarily disconnected. The dashboard falls back to demo mode automatically.
NFR3	Portability	The system must run on any machine with Python 3 and a browser, without requiring cloud services or external databases.
NFR4	Usability	The dashboard must be accessible and readable on both desktop and mobile screens without horizontal scrolling.
NFR5	Scalability	The database schema must allow the addition of new sensor types without requiring structural changes to the existing codebase.
NFR6	Alert Responsiveness	The dashboard must detect and visually display a threshold breach (alert banner, status change) as soon as the corresponding reading is fetched, with no noticeable delay beyond the normal refresh cycle.
NFR7	Data Persistence	All sensor readings must be stored permanently in the local database and must not be lost when the server is restarted.
NFR8	Notification Latency	The system must deliver a Telegram alert within a few seconds of a threshold breach being detected, so the user can react promptly.
NFR9	Actuator Response Time	The relay must respond to a remote activation/deactivation command within 1–2 seconds of the command being sent from the dashboard.

NFR10	Maintainability	The frontend, backend, and firmware are structured as separate, independently modifiable components (dashboard, Flask API, MQTT subscriber, ESP8266 firmware), allowing each part to be updated without affecting the others.
--------------	------------------------	---

3.5 Technology Analysis

3.5.1 Hardware Selection

For FR1 (sensor data acquisition) and FR2 (data transmission), we needed a development board with WiFi capability and enough analog/digital pins to connect all four sensors. We compared three options:

Board	Built-in WiFi	Affordability	Ease of Use
Arduino UNO	No (needs shield)	Medium	High
ESP8266 (NodeMCU)	Yes	Excellent ✓	High
Raspberry Pi 3	Yes	Poor	Medium

We chose the ESP8266 (NodeMCU v3). It has built-in WiFi which is essential for FR2, it is low cost which satisfies NFR6, and it is well-supported in the Arduino ecosystem. The Arduino UNO was rejected because it needs an external WiFi shield, and the Raspberry Pi is too expensive and too powerful for what we need.

3.5.2 Communication Protocol

For FR2, we needed to choose how the ESP8266 sends data to the server. The main candidates were HTTP (REST), MQTT, and CoAP.

Protocol	Pattern	Resource Usage	Suitability
HTTP/REST	Client-Server (request/response)	High (full HTTP headers)	Good for web APIs, not ideal for IoT
MQTT	Publish-Subscribe	Very low overhead	Ideal for IoT — chosen ✓
CoAP	Client-Server (UDP)	Low	Good for constrained networks, less common

MQTT was chosen because it is the standard protocol for IoT sensor communication. It uses a publish-subscribe model, meaning the ESP8266 publishes data to topics and any subscriber (our Python script) receives it automatically. This satisfies NFR1 because it is event-driven rather than polling-based. The broker used is Mosquitto, running locally on the server machine.

3.5.3 User Interface

For FR4 and FR5, we needed to decide on the type of user interface. The options we considered were a mobile native app, a Telegram bot, and a web dashboard.

- **Mobile app:** would require separate development for iOS and Android, out of scope for this project.
- **Telegram bot:** simple to implement but limited in terms of data visualization and historical charts. Ultimately adopted separately as a notification channel (FR7), rather than as the primary interface.
- **Web dashboard:** accessible from any device with a browser, fully customizable, supports interactive charts. **Chosen. ✓**

The web dashboard was built using HTML, CSS, and JavaScript with Chart.js for the charts. Flask serves the dashboard and exposes three API endpoints: `/api/latest` and `/api/history`, called by the frontend every 5 seconds to update the live and historical views (FR4, FR5), and `/api/relay`, used to send remote actuator commands to the ESP8266 (FR8).

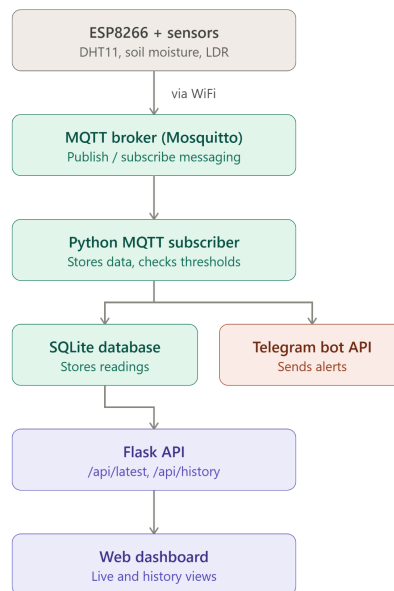
4. Design Phase

4.1 System Architecture

The system is organized in three main layers as shown below:

Data Acquisition Layer → Communication Layer → Application Layer

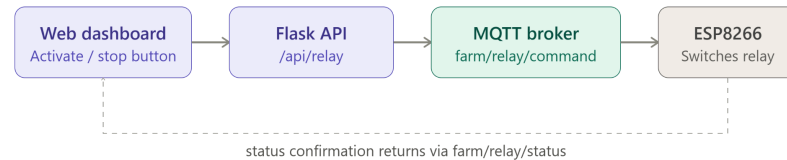
Data flows through the system in two directions: sensor data flows from the device to the application layer for monitoring, while actuator commands flow from the application layer back to the device for control.



Monitoring flow (sensor data → dashboard):

- ESP8266 reads sensors every 10 seconds (DHT11 for temperature/humidity, analog pin for soil moisture, analog pin for light via LDR).
- ESP8266 connects to the Mosquitto broker on the local network and publishes one MQTT message per sensor to its respective topic.

- Python subscriber (`subscriber.py`) receives the messages, inserts records into the SQLite database, and checks each reading against predefined thresholds; if a threshold is breached, it sends a Telegram alert (FR7).
- Flask application (`app.py`) exposes `/api/latest` and `/api/history` endpoints that query the database.
- The web dashboard (`dashboard.html`) fetches `/api/latest` every 5 seconds and updates KPI cards, charts, and alert banners (FR4, FR5, FR6). On page load it also fetches `/api/history` to pre-populate charts with the last 60 readings.



•

Actuation flow (dashboard → device, FR8):

- The user triggers the relay via a button on the dashboard.
- The dashboard sends a POST request to the `/api/relay` endpoint.
- Flask publishes an MQTT command ("ON"/"OFF") to the topic `farm/relay/command`.
- The ESP8266, subscribed to this topic, receives the command and switches the relay state accordingly, activating or deactivating the connected actuator.
- The ESP8266 publishes a confirmation on `farm/relay/status`.

4.2 Database Design

The database uses a single table to store all sensor readings. The design was chosen to keep the schema simple and flexible if a new sensor topic is added in the future, no schema change is needed.

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique row identifier
topic	TEXT	NOT NULL	MQTT topic (like: farm/temperature)
value	TEXT	NOT NULL	Sensor reading as string
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time the reading was stored

One record is inserted for each MQTT message received. The Flask API reconstructs structured readings by querying each topic separately and it merges them by timestamp.

4.3 Data Exchange Format

The ESP8266 publishes JSON payloads to each sensor topic. The format used is:

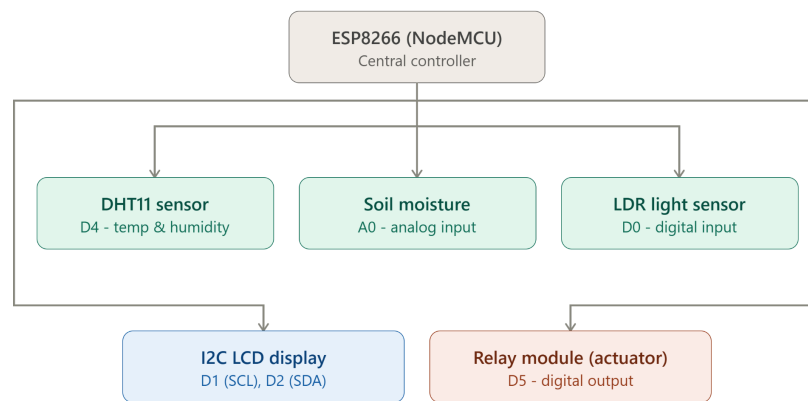
```
{"value": 24.5}
```

Each sensor has its own topic (`farm/temperature`, `farm/humidity`, `farm/moisture`, `farm/light`) so the subscriber can route messages independently. When the hardware is not yet available, the seed script inserts test data directly into the database using the same schema, so the rest of the stack behaves identically.

For actuator control (FR8), a simpler plain-text format is used instead of JSON: the topic `farm/relay/command` carries the literal string "ON" or "OFF". Upon receiving this command, the ESP8266 switches a relay module connected to digital pin D5. The relay's switching side is wired to an LED (with a 220Ω current-limiting resistor), which visually represents the activation and deactivation of a real irrigation pump or valve – a physical pump was not available at this stage of the project, so the LED serves as a stand-in load to demonstrate the complete remote-switching logic. The ESP8266 publishes a corresponding confirmation on `farm/relay/status` using the same plain-text format, once the relay state has changed.

4.4 Hardware Wiring Design

The ESP8266 (NodeMCU v3) is connected to sensors as follows:



Sensor	Pin on ESP8266	Signal Type	MQTT Topic
DHT11 (Temperature)	D4 (GPIO2)	Digital	farm/temperature
DHT11 (Humidity)	D4 (GPIO2)	Digital (same pin)	farm/humidity
Soil Moisture Sensor	A0 (ADC)	Analog	farm/moisture
LDR Light Sensor	D0	Digital	farm/light
Relay Module	D5	Digital output	farm/relay/command (subscribed), farm/relay/status (published)

The DHT11 sensor provides both temperature and humidity readings over a single digital data line (D4). The soil moisture sensor is read through the ESP8266's single analog input (A0). The LDR light sensor is read digitally via D0, detecting relative brightness rather than a calibrated light intensity value.

For actuation, the relay module's control pin (S) is connected to D5. Unlike the sensors, this pin is configured as an output rather than an input: the ESP8266 drives it HIGH or LOW based on commands received over MQTT (see Section 4.3), switching the relay to activate or deactivate the connected load (an LED simulating the irrigation pump, see Section 4.3). Power for all components is supplied via the ESP8266's 3.3V pin during development, with the board itself powered through USB.

4.5 Software Flowcharts

4.5.1 ESP8266 Data Collection and Transmission (FR1, FR2)

The Arduino code on the ESP8266 follows this logic:

- Start: connect to WiFi and MQTT broker.
- Loop every 2 seconds: read DHT11 (temperature, humidity) → validate reading (retry once if invalid) → read soil moisture via A0 → read light level via digital pin D0 → update the LCD display with current values.
- Every 10 seconds (publish interval): publish all four readings as JSON payloads to their respective topics (`farm/temperature`, `farm/humidity`, `farm/moisture`, `farm/light`).
- If WiFi or MQTT connection is lost: reconnect automatically before resuming normal operation.

4.5.2 Python MQTT Subscriber (FR3)-Data Storage and Alerting (FR3, FR7)

The subscriber (`subscriber.py`) is a single MQTT client that handles both data storage and threshold-based alerting within the same message handler:

- On connect: subscribe to four explicit topics `farm/temperature`, `farm/humidity`, `farm/moisture`, `farm/light`.
- On message: parse the JSON payload, extract the `value` field, and convert it to a float.
- Insert a new record into the SQLite database (`sensor_data` table) with the topic, value, and current timestamp.
- Compare the value against threshold constants defined locally in the subscriber (duplicated from the thresholds used on the dashboard, since the two currently run as independent components).
- If a threshold is breached, construct a descriptive alert message and send it via the Telegram Bot API.
- The entire process (storage, comparison, and alerting) is wrapped in a single try/except block; if any step fails (e.g. malformed payload, database error, network issue), the error is logged to the console and the message is skipped, without stopping the subscriber.

4.5.3 Dashboard Alert Algorithm (FR6)

- On each data fetch: compare temperature, humidity, moisture, and light against threshold constants.
- If value exceeds `dangerHigh` or falls below `dangerLow`: mark as "danger" → **red alert badge**.
- If value exceeds `warnHigh` or falls below `warnLow`: mark as "warn" → **orange alert badge**.
- Farm Status is determined by the worst level among all four sensors.
- Alert banners are updated dynamically without any page reload.

4.5.4 Remote Actuator Control (FR8)

The user triggers "Activate" or "Stop" on the dashboard, sending a POST request to the Flask `/api/relay` endpoint with the command ("ON"/"OFF").

Flask publishes the command as a plain-text MQTT message to `farm/relay/command`.

The ESP8266, subscribed to this topic, receives the command in its MQTT callback and sets the relay control pin (D5) HIGH or LOW accordingly.

The relay switches its state, activating or deactivating the connected load (LED simulating the pump).

The ESP8266 publishes a status confirmation to `farm/relay/status`.

4.6 Digital Twin Concept

In this system, the digital twin of the physical farm environment is represented by the SQLite database combined with the live dashboard. Each physical sensor reading on the ESP8266 corresponds to a record in the database, and the dashboard presents a real-time virtual representation of the farm state.

The dashboard KPI cards show the last known value of each physical sensor, while the line charts show their evolution over time. The Farm Status indicator combines all sensor states into a single health indicator that represents the overall condition of the physical environment.

Beyond passively reflecting sensor state, the digital twin also supports control: the actuator (relay) state can be changed directly from the dashboard, and this action is mirrored back to the physical device. This closes the loop between the virtual and physical systems — the dashboard not only represents the farm's current condition but also allows the user to influence it remotely, which is a defining characteristic of a true digital twin rather than a simple monitoring display.

4.7 Dashboard Mock-up

The dashboard is organized in two pages: Live and History.

The Live page layout (from top to bottom):

- Navigation bar with SmartFarm brand, page tabs, and live status indicator.
- Alert banner (hidden when all sensors are within specified safe range).
- Farm Status bar showing overall health.
- Actuator Control card, allowing the user to remotely activate or deactivate the irrigation relay (Activate/Stop buttons), with a status line showing the last command sent.
- Four KPI cards: Temperature, Humidity, Soil Moisture, Light — each with current value and delta from the last reading.
- 2×2 grid of line charts, each with a 10/30/60 point range selector.

The History page layout:

- Date range picker and sensor filter dropdown.
- Four average statistic cards.
- Four historical line charts, one per sensor.

5. Implementation Phase

5.1 System Construction

The implementation was split between the two team members. The hardware and Arduino side was handled on a part, while the software stack from another part (subscriber, database, Flask API, and dashboard).

5.1.1 Hardware Implementation

The ESP8266 was programmed using the Arduino IDE with the following libraries: `ESP8266WiFi`, `PubSubClient` for MQTT, `DHT` for the temperature/humidity sensor, and `LiquidCrystal_I2C` for the LCD display. The board connects to the local WiFi network on startup and immediately begins publishing sensor data to the Mosquitto broker every 10 seconds, while sensor readings and the LCD display are refreshed every 2 seconds.

A relay module was later added to support FR8 (remote actuator control). The ESP8266 subscribes to the topic `farm/relay/command` and switches a digital output pin (D5) HIGH or LOW based on received commands, driving the relay and, in turn, an LED used to simulate a physical irrigation pump.

During development, the hardware was tested independently by verifying serial monitor output before integrating with the MQTT pipeline. The sensor readings were validated against known reference values to check accuracy. The relay was similarly tested in isolation by publishing manual MQTT commands via `mosquitto_pub` and confirming the relay's physical switching (audible click and LED response) before connecting it to the dashboard.

5.1.2 Software Implementation

On the software side, development started with the database schema and the MQTT subscriber. A test data script was written to simulate realistic sensor readings so the dashboard could be developed and tested without needing the physical hardware at a first point.

The Flask backend was implemented with two endpoints. The `/api/latest` endpoint returns the most recent value for each sensor topic. The `/api/history` endpoint returns all readings within a date range, merging records by timestamp. Both endpoints return **JSON**.

The dashboard was built as a single HTML file with embedded CSS and JavaScript. `Chart.js` was used for the line charts. On page load, the history endpoint is called to fill the charts with the last 60 readings, so the user sees real data immediately rather than waiting for the 5-second polling cycle to accumulate points.

A third endpoint, `/api/relay`, accepts POST requests from the dashboard and publishes the corresponding command to the ESP8266 over MQTT, supporting remote actuator control (FR8).

5.2 Testing

Testing was done in stages:

- **Unit testing of the MQTT subscriber:** messages were manually published using the `mosquitto_pub` command and the database was inspected to verify correct insertion.
- **API testing:** the Flask endpoints were tested using a browser directly (visiting `/api/latest` and `/api/history`) to verify the JSON output format.
- **Integration testing:** with the full stack running (Mosquitto + subscriber + Flask), the ESP8266 was connected and data was verified to appear on the dashboard within the expected 5-second window.
- **Fallback testing:** the Flask server was stopped while the dashboard was open to verify that the demo mode activates correctly and the page does not break.

- **Actuator testing:** the relay was first tested manually via `mosquitto_pub` commands sent directly to `farm/relay/command`, confirming the ESP8266 received the command and switched the relay. The dashboard's Activate/Stop buttons were then tested end-to-end, confirming the full chain (dashboard → Flask → MQTT → ESP8266 → relay) functioned correctly.
- **Telegram alert testing:** threshold breaches were simulated by physically manipulating sensor readings (covering the soil moisture sensor, raising ambient temperature) to confirm that the subscriber correctly detected the breach and delivered a Telegram notification.

5.3 Known Limitations and Future Work

The current implementation has some limitations that could be addressed in future versions:

- There is no authentication on the dashboard or the API endpoints. Anyone on the local network can access the data or trigger the actuator.
- The system currently runs on a local network only. Adding cloud integration (for example, publishing to an MQTT cloud broker) would make remote monitoring possible from outside the local WiFi network.
- The database grows indefinitely. A data retention policy (for example, automatically deleting readings older than 30 days) should be added for long-term deployments.
- Threshold values are currently duplicated independently in the subscriber (for Telegram alerts) and in the dashboard (for visual alerts), rather than being defined in a single shared configuration. Updating a threshold currently requires changing it in both places.
- The actuator (FR8) currently drives an LED as a stand-in for a real irrigation pump or valve, since physical actuation hardware was not available at this stage. Additionally, the dashboard does not yet display the actuator's live ON/OFF state only the last command sent so a future improvement would be to subscribe to `farm/relay/status` on the frontend and reflect the confirmed state in real time.
- The ESP8266 has only one true analog input (A0), currently used by the soil moisture sensor. Adding further analog sensors in the future would require a multiplexer or external ADC.

6. Conclusion

This project demonstrates how a complete IoT monitoring and control system can be built using low-cost hardware and open-source software. The ESP8266 proved to be a suitable choice for this kind of application; it is affordable, easy to program, and handles WiFi communication natively.

The MQTT protocol worked well for both sensor data transmission and actuator control. Its publish-subscribe model means the ESP8266 does not need to know anything about who is consuming its sensor data, and conversely does not need to know who issued a command it simply reacts to messages on the topics it subscribes to. This symmetry made it straightforward to extend the system from a one-way monitoring pipeline into a two-way system capable of both sensing and acting on the environment.

The web dashboard achieved the main goal of providing a clear and readable view of farm conditions in real time. The use of threshold-based alerts and the Farm Status indicator make it easy to spot problems quickly without having to read individual sensor values, while the Telegram notification channel extends this awareness beyond the dashboard, notifying the user even when they are not actively watching it. The addition of remote actuator control (FR8) moved the system beyond passive monitoring, allowing the user to respond directly to detected conditions for example, triggering irrigation closing the loop between observation and action, which is central to the digital twin concept introduced in Section 4.6.

Working in two people with a hardware/software split was effective but required careful coordination, especially around the MQTT message format and the database schema. The decision to write a seed script early on allowed the software side to progress independently without waiting for the physical hardware. Working within the constraints of available components for instance, using an LED to simulate a real irrigation pump also reflected a realistic engineering process, where design decisions are shaped as much by available resources as by ideal specifications.